

Building a RAG System for Document Retrieval

A practical guide to implementing Retrieval-Augmented Generation for PDF document querying

What is Retrieval-Augmented Generation?



RAG combines document retrieval with generative AI to answer questions using your own data. Instead of relying solely on a language model's training data, RAG searches through your documents to find relevant context first, then generates answers grounded in that evidence.

By encoding document content into vector embeddings, the system enables [semantic search](#) that understands meaning rather than just matching keywords.

Five Core Components

01

PDF Processing

Extract raw text content from documents while preserving structure

02

Text Chunking

Split content into semantically coherent segments for precision

03

Vector Store Creation

Generate embeddings using HuggingFace models and FAISS indexing

04

Retriever Setup

Configure interface to surface most relevant document chunks

05

System Evaluation

Assess retrieval quality against defined benchmarks

Document Preprocessing Pipeline



Load PDF

PyPDFLoader extracts text while preserving page structure



Split Text

RecursiveCharacterTextSplitter creates overlapping chunks



Clean Content

Remove tab characters and normalize whitespace

Overlapping chunks ensure contextual continuity — sentences spanning boundaries aren't lost. This preserves meaning across adjacent segments.

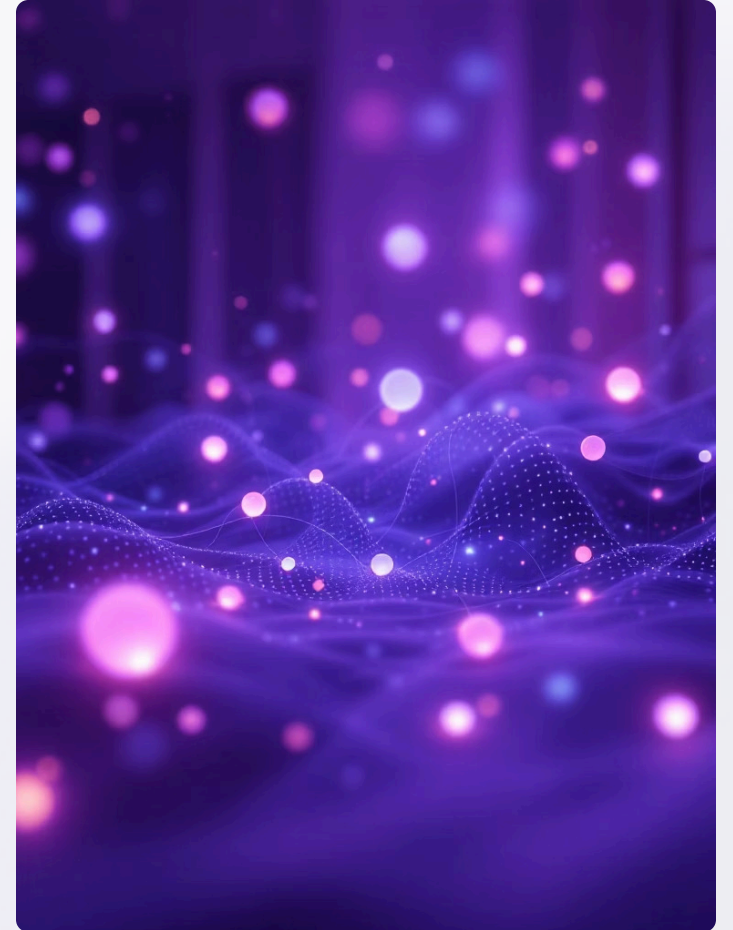
Vector Store Architecture

Embedding Generation

Each cleaned text chunk passes through HuggingFace's embedding model, transforming raw text into high-dimensional vectors that capture semantic meaning.

FAISS Indexing

Vectors are indexed using Facebook AI Similarity Search, optimized for fast nearest-neighbor lookups in large vector spaces. Similarity computations happen in milliseconds regardless of document size.



Retrieval Strategy



Top-K Retrieval

Returns **top 2 most similar chunks** per query



Context Window

Concise retrieval reduces noise for downstream models



Configurable

Adjust retrieval count based on document verbosity

The retriever searches for semantically aligned chunks, which can then be passed to a language model for grounded, document-specific answers.



Key Features



Modular Design

encode_pdf function encapsulates loading, cleaning, embedding, and indexing behind one interface. Caller supplies only file path and parameters.



Configurable Chunking

Chunk size and overlap exposed as parameters. Tune for documents with varying structure — from dense academic papers to sparse FAQ pages.



Efficient Search

FAISS optimized for high-dimensional vector spaces. Scales linearly with chunks, not exponentially. Handles growing document collections gracefully.



Semantic Understanding

State-of-the-art embeddings trained on diverse corpora. Generalizes across domains. Captures meaning beyond exact phrasing matches.

Benefits of This Approach



Scalability

Processes large files in chunks without memory constraints. FAISS index grows linearly with chunk count, not exponentially.



Flexibility

Adjustable parameters adapt to document types and query patterns without code changes.



Efficiency

FAISS uses approximate nearest-neighbor search with optimized indexing. Low-latency lookups across hundreds of thousands of vectors.



Semantic Search

Understands meaning beyond keywords. Query about "carbon emissions" surfaces passages about "greenhouse gases" without lexical overlap.

Ready for Production

What Makes This Pipeline Strong

- Clean separation of concerns with configurable parameters
- Retrieval mechanism that scales with document growth
- Combines state-of-the-art embeddings with FAISS indexing
- Transforms static PDFs into interactive, queryable knowledge



For teams building document-grounded question-answering tools — internal knowledge bases, legal review, or technical support — this pipeline offers a practical, extensible starting point.